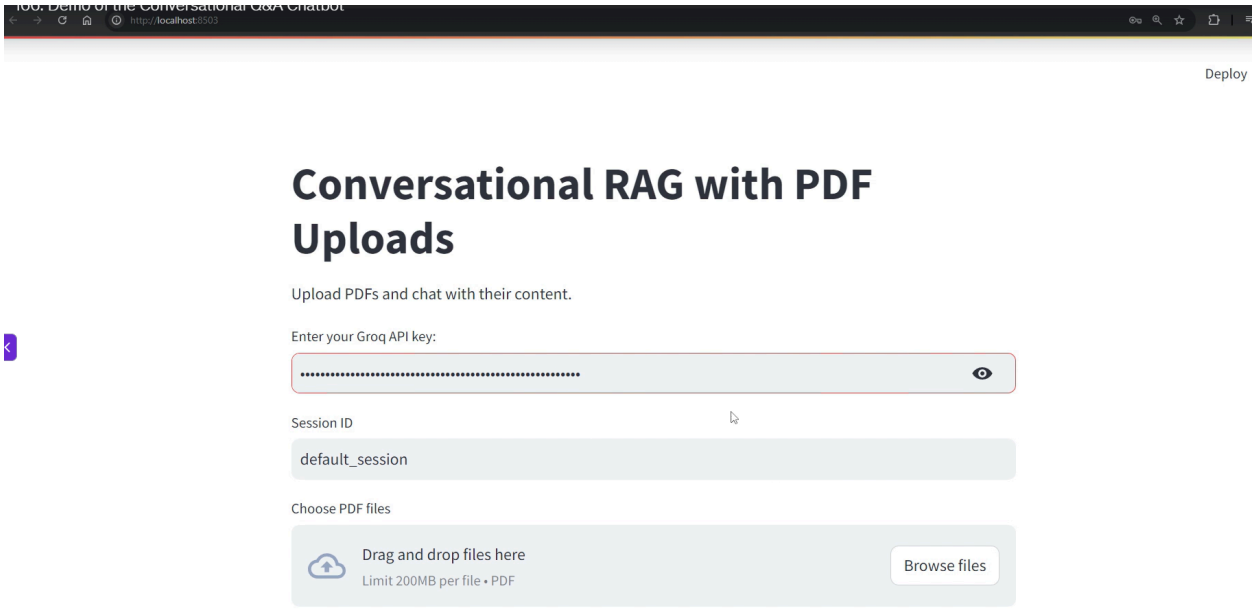

here is a project where you will upload pdf files and from there model will answer what ever the question you ask, along with all the previous summary of the questions you asked before.

Refer section29 document for all terminology

This is a **Streamlit app that builds a conversational RAG (Retrieval-Augmented Generation) chatbot over PDFs with memory (chat history)** using tools like LangChain, Hugging Face, and Groq.

Let's walk through it step-by-step in a way that actually *clicks*.

🧠 High-level flow (what this app does)

1. Upload PDFs
2. Convert them into embeddings
3. Store in a vector database
4. Ask questions
5. Retrieve relevant chunks
6. Answer using an LLM
7. Maintain chat history

👉 This is a **Conversational RAG pipeline**

🧱 1. Imports (building blocks)

You're pulling components from LangChain:

- `create_retrieval_chain` → main RAG pipeline
- `Chroma` → vector database
- `ChatGroq` → LLM via Groq
- `HuggingFaceEmbeddings` → embeddings from Hugging Face
- `PyPDFLoader` → reads PDFs

🔐 2. Environment setup

load_dotenv()
os.environ['HF_TOKEN']=os.getenv("HF_TOKEN")
embeddings=HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")

- Loads Hugging Face token
- Initializes embedding model (`all-MiniLM-L6-v2`)
  👉 Converts text → vectors

🖥️ 3. Streamlit UI

st.title(...)
api_key=st.text_input(...)

- UI for:
  - Groq API key
  - PDF upload
  - User question

⚡ 4. LLM setup (Groq)

llm=ChatGroq(groq_api_key=api_key, model_name="Gemma2-9b-It")

- Uses Groq to run the model
  👉 This is your **answer generator**

📂 5. PDF upload + loading

loader=PyPDFLoader(temppdf)
docs=loader.load()

- Saves uploaded file
- Extracts text from PDF

✂️ 6. Text splitting

text_splitter = RecursiveCharacterTextSplitter(chunk_size=5000, chunk_overlap=500)
splits = text_splitter.split_documents(documents)

👉 Why?

- LLMs can't handle huge documents
- So you break them into chunks

🧮 7. Embeddings + vector store

vectorstore = Chroma.from_documents(documents=splits, embedding=embeddings)
retriever = vectorstore.as_retriever()

- Converts chunks → embeddings
- Stores in **Chroma DB**

- Retriever finds relevant chunks

👉 This is the **search engine part**

🧠 8. History-aware retriever (very important)

history_aware_retriever=create_history_aware_retriever(...)

What it does:

If user asks:

> “What about its pricing?”

It rewrites to:

> “What is the pricing of X (from previous context)?”

👉 Uses chat history to **fix ambiguous questions**

💬 9. Prompt for answering

system_prompt = "Use context to answer..."
qa_prompt = ChatPromptTemplate.from_messages(...)

- Tells LLM:
  - Use retrieved context
  - Be concise
  - Say “don’t know” if needed

🔗 10. RAG chain

question_answer_chain=create_stuff_documents_chain(llm,qa_prompt)
rag_chain=create_retrieval_chain(history_aware_retriever,question_answer_chain)

👉 This connects:

- Retriever → gets context
- LLM → generates answer

🧠 11. Chat history management

st.session_state.store={}

- Stores conversation per session

def get_session_history(session):
    ...

👉 Returns chat history for a session

🔁 12. Add memory to chain

RunnableWithMessageHistory(...)

This is the **magic**:

- Injects chat history into:
    - retriever
    - prompt
- Saves responses back into memory

👉 Makes it **conversational**

❓ 13. Asking questions

```
response = conversational_rag_chain.invoke(
    {"input": user_input},
    config={"configurable": {"session_id":session_id}}
)
```

Flow when user asks:

User question
↓
Rewrite using history
↓
Search vector DB
↓
Get relevant chunks
↓
Send to LLM (Groq)
↓
Generate answer
↓
Store in history

🖥️ 14. Output

```
st.write("Assistant:", response['answer'])
st.write("Chat History:", session_history.messages)
```

🧠 Key concepts (super important)

✅ This app combines:

- **RAG** → retrieval + generation
- **Memory** → chat history
- **Embeddings** → semantic search
- **LLM** → answer generation

⚠️ Subtle issues / improvements

A few things to watch:

1. File overwrite issue

temppdf="./temp.pdf"

- Multiple uploads overwrite same file

👉 Fix: use unique filenames

---

2. Large chunk size

chunk_size=5000

- Might be too big → slower retrieval

👉 Usually 500–1500 works better

---

3. No persistence

- Chroma DB is in-memory
  👉 Data lost on refresh

🧠 Final mental model

This app is basically:

👉 **ChatGPT + your PDFs + memory**

Where:

- Hugging Face → embeddings

- Groq → fast LLM
- LangChain → glue everything

If you want, I can:

- Draw an **architecture diagram**
- Or simplify this into **20 lines of minimal code** so it's easier to learn

Here's a **minimal ~20-line version** of your RAG + PDF + chat history app. I stripped it down to the core idea: upload PDF → embed → retrieve → ask LLM → remember chat.

This keeps the essence of your original code using LangChain, Groq, and Hugging Face embeddings.

🧠 Minimal RAG Chat with PDF (Streamlit)

```
import streamlit as st
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_chroma import Chroma
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_groq import ChatGroq
from langchain.chains import create_retrieval_chain, create_stuff_documents_chain
from langchain_core.prompts import ChatPromptTemplate

st.title("Mini RAG PDF Chat")

api_key = st.text_input("Groq API Key", type="password")

uploaded_file = st.file_uploader("Upload PDF", type="pdf")
question = st.text_input("Ask a question")

if api_key and uploaded_file:

    # Load PDF
    with open("temp.pdf", "wb") as f:
        f.write(uploaded_file.read())
    docs = PyPDFLoader("temp.pdf").load()
```

```
    # Split
    chunks = RecursiveCharacterTextSplitter(chunk_size=1000,
chunk_overlap=200).split_documents(docs)

    # Embeddings + Vector DB
    embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
    db = Chroma.from_documents(chunks, embeddings)
    retriever = db.as_retriever()

    # LLM
    llm = ChatGroq(groq_api_key=api_key, model_name="Gemma2-9b-It")

    # Prompt
    prompt = ChatPromptTemplate.from_template(
        "Answer using context:\n{context}\nQuestion: {input}"
    )

    qa_chain = create_stuff_documents_chain(llm, prompt)
    rag_chain = create_retrieval_chain(retriever, qa_chain)

    if question:
        result = rag_chain.invoke({"input": question})
        st.write(result["answer"])
```

🧠 What you removed (intentionally)

This simplified version removes:

- ❌ Chat history (memory)
- ❌ History-aware retriever
- ❌ Session management
- ❌ Complex prompt engineering
- ❌ Multiple PDFs handling
- ❌ Environment variables
- ❌ Persistence of vector DB

🧩 What you kept (core RAG loop)

PDF → chunks → embeddings → vector DB → retrieve → LLM → answer

That's the **entire RAG system in its simplest form**.

🚀 If you want next step

I can upgrade this step-by-step into:

1. 🔁 Add chat memory (like ChatGPT)
2. 📚 Multi-PDF support
3. 🧠 History-aware retrieval
4. 💾 Persistent vector DB
5. ⚡ Swap Groq ↔ Ollama locally

Just tell me 👍

`MessagesPlaceholder` explained

`MessagesPlaceholder` is a LangChain prompt helper used inside a `ChatPromptTemplate`.

- It represents a slot where the model's chat history is inserted.
- In your code, it is used as part of the prompt structure:
  - `MessagesPlaceholder("chat_history")`

That means:
- when the chain runs, it will fill that placeholder with the previous conversation messages stored under the key `chat_history`
- the prompt becomes:
  - system instruction
  - previous chat history
  - current user question

So `MessagesPlaceholder` is not a text string itself — it's a template marker telling LangChain: "inject the saved chat history here before sending the prompt to the model."